

Vigno Smart Class — License System

Document 2 of 2 · **Detailed Technology & Step-by-Step Implementation**

Exact stack, signed-token format, database, APIs, security & build plan

Read after Document 1 — High-Level Design

1. Technology Stack (per component)

MERN core, plus the specific tools that make the license system work.

Component	Technology	Why
Web app	React + Vite, Tailwind, React Query	UI, in-browser secure viewer, cached data
API server	Node.js + Express	Auth, ownership checks, License Authority
Database	MongoDB + Mongoose (Atlas)	users, purchases, licenses, devices
Login tokens	JWT (jsonwebtoken) in httpOnly cookies	Proves who is logged in
License tokens	JWS / signed JWT (RS256 or ES256)	Signed, tamper-proof permission
Key storage	AWS KMS (or HashiCorp Vault)	Holds the private signing key safely
File storage + CDN	AWS S3 (SSE-KMS) + CloudFront signed URLs	Encrypted files, short-lived delivery
Video DRM (optional)	Widevine / FairPlay via Mux or VdoCipher	Studio-grade video protection
Payments	Razorpay (order + signature + webhook)	Trigger to issue a license
Watermarking	pdf-lib / ffmpeg overlay	Stamp buyer id to trace leaks
Launcher (downloads only)	Electron or Tauri + native crypto	Verify license & decrypt on device
Security middleware	Helmet, express-rate-limit, Zod, CORS	Harden every endpoint

2. The License Token (signed JWT / JWS)

A compact, signed token. Header + payload are readable; the signature proves our server made it and nobody changed it.

Field	Example	Meaning
jti	lic_8f2a...	Unique license id (for revocation)
sub	user_123	Owner account
cid	content_777	Content / game id
typ	"stream" "download"	Which lane
dev	device fingerprint hash	Bound device (downloads only)
iat / exp	issued / expiry time	Short life + refresh
alg	ES256 / RS256	Signing algorithm

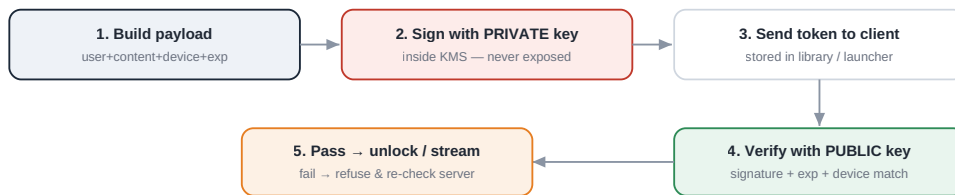
Example (decoded)

```
// header
{ "alg": "ES256", "typ": "JWT", "kid": "vigno-key-2026" }
// payload
{ "jti":"lic_8f2a", "sub":"user_123", "cid":"content_777",
  "typ":"download", "dev":"d3f1...b9", "iat":1780000000, "exp":1780604800 }
// signature
<ECDSA signature over header.payload, made with the PRIVATE key>
```

Edit any field → the signature no longer matches → verification fails. No private key → no valid signature can be created.

3. Signing & Verification Flow

The server signs once; every client (browser viewer or launcher) verifies with the public key.



4. Database Schema (license-related)

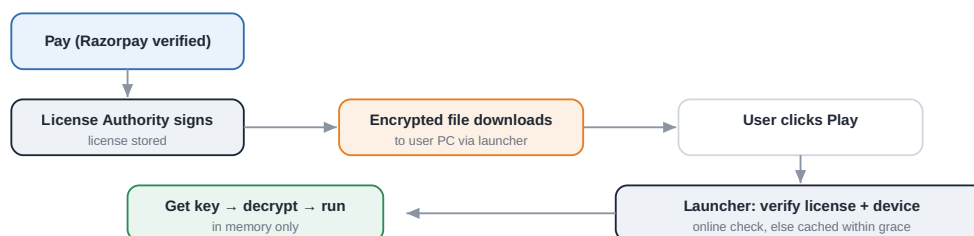
Collection	Key fields	Notes
users	_id, email, passwordHash, role, twoFAEnabled	Identity
contents	_id, title, type(stream/download), s3Key, isPaid, price	Catalog item
purchases	_id, userId, contentId, amount, razorpayPaymentId, status	Proof of payment
licenses	_id(jti), userId, contentId, type, status(active/revoked), issuedAt, expiresAt	Source of truth for ownership
devices	_id, userId, fingerprint, name, boundLicenseIds[], lastSeen	For downloadable software
auditlogs	actorId, action, targetId, ip, time	Security trail

Indexes: `licenses(userId, contentId)`, `licenses.status`, `devices(userId, fingerprint)` — instant ownership lookups.

5. API Endpoints

Method & Path	Who	Does
POST /auth/login	Public	Login, set JWT cookie
POST /payments/verify	User	Confirm payment → trigger license issue
POST /licenses/issue	Server (internal)	Sign & store a license after verified payment
GET /licenses/mine	User	List the user's licenses (library)
POST /licenses/verify	User / Launcher	Re-check a license is active + not expired
GET /content/:id/stream-url	User	Ownership check → 60s signed URL (study material)
POST /devices/register	User / Launcher	Bind a device fingerprint
POST /content/:id/key	Launcher	Verify license+device → return decryption key
POST /admin/licenses/:id/revoke	Admin	Revoke (refund / fraud)
GET /.well-known/vigno-public-key	Public	Publish the public verify key

6. End-to-End Sequence (buy a game → play it)



7. Downloadable Software: Launcher & Device Binding

For .exe / games only. This is the part a website alone cannot do — it needs a small desktop app, exactly like the Steam/console client.

Launcher responsibilities

- └─ Login → get JWT + user identity
- └─ Register device → fingerprint (CPU/MB/OS hash)
- └─ Download → keep file ENCRYPTED on disk
- └─ On Play
 - └─ verify license (online; cached within grace)
 - └─ check dev in license == this device
 - └─ request decryption key from server
 - └─ decrypt in memory → launch game
- └─ On close → only encrypted file remains

Device binding means a copied encrypted file won't unlock on another machine — its fingerprint won't match the license. This mirrors a console's "home device" activation.

8. Security in Detail

8.1 Key management

Topic	Approach
Private signing key	Generated & stored in AWS KMS / Vault; signing happens via KMS API — key never leaves
Public key	Published at a fixed URL + bundled in launcher for offline verify
Key rotation	Use <code>kid</code> (key id) in the token header; rotate yearly, keep old public keys to verify old tokens

8.2 Threats → mitigations

Attack	Mitigation
Forge a license	Impossible without private key (public-key crypto)
Edit license fields	Signature breaks → rejected
Replay an old/ expired license	Short <code>exp</code> + server re-check + revocation list
Copy file to another PC	Device fingerprint mismatch → no key issued
Share a stream link	60-second signed URL already expired
Patch the launcher ("always true")	Server-issued decryption key is required to decrypt — patching the check doesn't reveal the key; add anti-tamper + integrity checks
Stolen account	2FA + new-device alerts + revoke sessions
Leaked PDF/video	Per-user watermark traces the buyer
Brute force / abuse	Rate limiting + WAF + audit logs

Honest note: no client-side protection is unbreakable (even Denuvo was bypassed). The realistic goals are: make casual copying useless, bind ownership to the account, and trace any leak — which this design achieves.

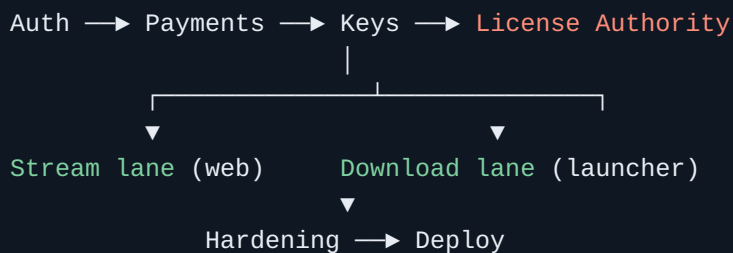
9. Step-by-Step Implementation Plan

Build in phases; each is testable before the next.

1. **Foundation** — MERN setup, MongoDB Atlas, auth (JWT cookies, bcrypt, RBAC), the content tree & admin CMS.
2. **Payments** — Razorpay order + signature verify + webhook; write a `purchases` record on success.
3. **Key setup** — create signing keypair in KMS; publish the public key endpoint.
4. **License Authority** — `/licenses/issue` (sign after verified payment), `/licenses/verify`, `/licenses/mine`, store in `licenses`.
5. **Study-material lane** — encrypted S3 upload; `/content/:id/stream-url` ownership check → CloudFront signed URL; in-browser viewer with download disabled; watermarking.
6. **Revocation** — admin revoke endpoint + check status on every verify (handles refunds/fraud).
7. **Download lane (if selling software)** — build the launcher (Electron/Tauri): login, device register, encrypted download, verify license + device, fetch key, decrypt & run; offline grace window.
8. **Optional video DRM** — integrate Widevine/FairPlay via Mux or VdoCipher for premium video.
9. **Hardening** — Helmet, rate limits, input validation, CORS, audit logs, key rotation plan.
10. **Testing & deploy** — unit/integration tests, a security review, then deploy (frontend CDN, backend PM2, Atlas, KMS).

Build dependency order

Phase tree



10. Go-Live Security Checklist

Area	Must-have before launch
Transport	HTTPS/TLS on all endpoints
Keys	Private key only in KMS; rotation + <code>kid</code> in place
Licenses	Short expiry, server re-check, revocation working
Files	S3 SSE encryption; no public bucket; signed URLs only
Accounts	bcrypt, optional 2FA, new-device alerts
App	Helmet, rate-limit, Zod validation, CORS locked
Trace	Watermarking live; audit logs capturing admin/license actions
Payments	Signature + webhook both verified before issuing license

Together, Documents 1 (design) and 2 (this) give you the full blueprint: the console-style license model, the exact technologies, the signed-token mechanics, and a phased, testable build plan.